

CS3210 Technical Reference

ngmh

22/04/2026

1 Performance Instrumentation

1.1 Timing

```
/usr/bin/time -v ./a
```

1.2 perf

- `perf stat -e LLC-loads,LLC-load-misses,cycles,instructions -r 3 -- ./a`
- `perf record -- ./a`
- `perf report`
- Events include `L1-dcache-load-misses`, `L1-dcache-loads`, `L1-dcache-stores`

1.3 Slurm

- `srun --partition i7-7700 bash -c "lscpu; hostname"`

2 OpenMP

2.1 Compilation Command

```
g++ -fopenmp code.cpp
```

2.2 Work Sharing Constructs

- `omp parallel`: Specify this section of code to be parallelised
- `omp for`: Indicate this for loop to be parallelised, DO NOT NEST!
- `collapse(n)`: Combines `n` perfectly nested loops into one large iteration space, improves load balancing when the outer loop has few iterations
- `schedule(static, chunksize)`: Divide for loop into pieces of size `chunksize`, and specify `static` assignment (can also be `dynamic`)
- `omp sections`: Specify the start of a block of sections, where the sections can run in parallel
- `omp section`: Specifies a section block
- `omp single`: This block will only be run by a single thread

2.3 Synchronisation Constructs

- `omp master`: This block will only be run by the master thread
- `nowait`: Specify that this block does not include an implicit barrier at the end
- `omp barrier`: Provides synchronisation where all threads have to arrive first
- `omp critical`: Specifies a critical section that can only be executed by one thread at a time
- `omp atomic`: Specifies a memory location that must be updated atomically

2.4 Data Scope Attribute Clauses

- `shared(a, b)`: Mark variables to be shared among threads
- `private(i, j, k)`: Mark variables to be private to a thread
- `reduction(op:list)`: Each thread gets a private copy of the variable, initialized based on the operator (e.g., 0 for +), all private copies are combined into original global variable

2.5 Runtime Library Routines

- `omp_get_thread_num()`: Get the thread id for the current thread
- `omp_set_num_threads()`: Set the number of threads OpenMP should use
- `omp_get_max_threads()`: Get the maximum number of threads OpenMP can use
- `omp_get_num_threads()`: Get the number of threads that OpenMP is actually using

2.6 Examples

2.7 Matrix Multiplication

```
1 void mm(matrix a, matrix b, matrix res) {
2     int i, j, k;
3     #pragma omp parallel for shared(a, b, res) private (i, j, k)
4     for (i = 0; i < size; i++)
5         for (j = 0; j < size; j++)
6             for (k = 0; k < size; k++)
7                 res[i][j] += a[i][k] * b[k][j];
8 }
```

2.8 Sections

```
1 #pragma omp parallel
2 {
3     #pragma omp sections
4     {
5         #pragma omp section
6         {
7             work1();
8         }
9         #pragma omp section
10        {
11            work2();
12        }
13    }
14 }
```

2.9 Reduction

```
1 double dot = 0;
2 #pragma omp parallel for reduction(+:dot)
3 for (int i = 0; i < n; i++)
4     dot += a[i] * b[i];
```

3 CUDA

3.1 Compilation

`nvcc -arch native -o a a.cu`, can specify Compute Capability e.g. `-arch sm_70`

3.2 Function Specifiers

- `__device__`: Executes on device (GPU), can only be called from device
- `__global__`: Kernel, executes on device, needs execution configuration in `<<>>`, callable from both device and host
- `__host__`: Executes on host (CPU), can only be called from host, default specifier

3.3 Memory Specifiers

- `None`: Register, accessible by a single thread, might spill to local memory
- `__shared__`: Shared memory, accessible by all threads in block, statically assigned, fast
- `__device__`: Global memory, accessible by all threads
- `__constant__`: Constant memory, accessible by all threads, read-only
- `__texture__`: Texture memory, accessible by all threads, read-only
- `__managed__`: Unified memory, accessible by all threads and CPU

3.4 Memory Allocation

```
1 cudaMalloc(void **pointer, size_t nbytes);
2 cudaMemcpy(void *pointer, int value, size_t count);
3 cudaFree(void* pointer);
4
5 cudaMemcpy(void *dst, void *src, size_t nbytes, enum cudaMemcpyKind direction);
6 // enum cudaMemcpyKind
7 // cudaMemcpyHostToDevice, cudaMemcpyDeviceToHost, cudaMemcpyDeviceToDevice
```

3.5 Synchronisation

- `__syncthreads()`: Synchronise all threads in a block with a barrier
- `__syncwarp()`: Force threads within a warp to synchronise
- `cudaDeviceSynchronize()`: Call on host, wait for GPU to finish all tasks
- `atomicAdd(void* ptr, val)`: Perform atomic RMW on value at pointer

3.6 Indexing

- `gridDim.x/y/z`: Number of blocks in the grid
- `blockDim.x/y/z`: Number of threads in a block
- `gridIdx.x/y/z`: Index of current block
- `blockIdx.x/y/z`: Index of thread within block
- See this [link](#) for more details

3.7 Kernel Execution

<<Dg, Db, Ns, S>>>

- Dg: Grid dimensions (dim3)
- Db: Block dimensions (dim3)
- Ns: Shared memory size in bytes (size_t, optional, defaults to 0)
- S: Stream ID (cudaStream_t, optional, defaults to 0)

3.8 Examples

3.8.1 Matrix Multiplication

```
1  __global__ void matmul(float *A, float *B, float *C, int size) {
2      int row = blockIdx.y * blockDim.y + threadIdx.y;
3      int col = blockIdx.x * blockDim.x + threadIdx.x;
4      if (row < size && col < size) {
5          float sum = 0.0f;
6          for (int k = 0; k < size; k++) {
7              sum += A[row * size + k] * B[k * size + col];
8          }
9          C[row * size + col] = sum;
10     }
11 }
12
13 int main() {
14     int size = 1024; // Matrix dimension (size x size)
15     size_t bytes = size * size * sizeof(float);
16
17     float *h_A = (float*)malloc(bytes);
18     float *h_B = (float*)malloc(bytes);
19     float *h_C = (float*)malloc(bytes);
20
21     // (Initialize h_A and h_B with data here)
22
23     float *d_A, *d_B, *d_C;
24     cudaMalloc(&d_A, bytes);
25     cudaMalloc(&d_B, bytes);
26     cudaMalloc(&d_C, bytes);
27
28     cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
29     cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
30
31     // Db: Threads per block (typically 16x16 or 32x32)
32     dim3 Db(16, 16);
33     // Dg: Number of blocks in the grid (ceiling division)
34     dim3 Dg((size + Db.x - 1) / Db.x, (size + Db.y - 1) / Db.y);
35
36     matmul<<<Dg, Db>>>>(d_A, d_B, d_C, size);
37
38     cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);
39
40     cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
41     free(h_A); free(h_B); free(h_C);
42 }
```

3.9 Profiling

- Nsight Compute: [SLURM] `ncu --set full --clock-control none ./a`, profiling tool for CUDA kernels
- Nsight Systems: [SLURM] `nsys profile --cuda-event-trac=false -o report ./a`, visualise timeline of events

3.10 Streams

- Operations from different streams can execute concurrently
- Operations within same stream remain implicitly synchronised

```
1 std::vector<cudaStream_t> streams(STREAM_COUNT);
2 for (auto& stream : streams) {
3     cudaStreamCreate(&stream);
4 }
5
6 // Map chunks to streams using modulo
7 for (int p = 0; p < total_chunks; p++) {
8     cudaStream_t s = streams[p % STREAM_COUNT];
9
10    // Asynchronous H2D Copy
11    cudaMemcpyAsync(d_in + p*N, h_in + p*N, bytes, cudaMemcpyHostToDevice, s);
12
13    // Kernel Execution on stream
14    transform<<<blocks, threads, 0, s>>>(d_in + p*N, d_out + p*N);
15
16    // Asynchronous D2H Copy
17    cudaMemcpyAsync(h_out + p*N, d_out + p*N, bytes, cudaMemcpyDeviceToHost, s);
18 }
19 cudaDeviceSynchronize();
20 for (auto& s : streams) cudaStreamDestroy(s);
```

4 OpenMPI

4.1 Environment and Point-to-Point Communication

```
1 int MPI_Init(int* argc, char** argv[])
2 int MPI_Finalize(void)
3 int MPI_Abort(MPI_Comm comm, int errorCode)
4
5 // May be asynchronous, blocking, may be buffered
6 int MPI_Send(void* buf, int count, MPI_Datatype dt, int dest, int tag, MPI_Comm c);
7 int MPI_Recv(void* buf, int count, MPI_Datatype dt, int src, int tag, MPI_Comm c,
8             MPI_Status *status);
9 // Can use MPI_ANY_SOURCE and MPI_ANY_TAG
10 // Status is MPI_SOURCE, MPI_TAG, MPI_ERROR
11 int MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, int dest,
12                 int sendtag, void *recvbuf, int recvcount, MPI_Datatype recvtype, int source, int
13                 recvtag, MPI_Comm comm, MPI_Status *status)
14
15 // Basic Usage
16 MPI_Init(&argc, &argv);
17 int rank, size;
18 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
19 MPI_Comm_size(MPI_COMM_WORLD, &size);
20
21 MPI_Send(&msg, 1, MPI_INT, rank+1, 0, MPI_COMM_WORLD);
22 MPI_Recv(&msg, 1, MPI_INT, rank-1, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
```

```

20
21 if(rank == 0) MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
22 if(rank == 1) MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
23
24 MPI_Sendrecv(&send_val, 1, MPI_INT, right, 0, &recv_val, 1, MPI_INT, left, 0,
    MPI_COMM_WORLD, &status);
25
26 MPI_Finalize();

```

4.2 Communication Modes

```

1 // May be asynchronous, non-blocking, may be buffered
2 int MPI_Isend(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm
    comm, MPI_Request *request);
3 int MPI_Irecv(void *buf, int count, MPI_Datatype datatype, int source, int tag,
    MPI_Comm comm, MPI_Request *request);
4 int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)
5 int MPI_Wait(MPI_Request *request, MPI_Status *status)
6 int MPI_Waitall(int count, MPI_Request array_of_requests[], MPI_Status *
    array_of_statuses)
7
8 // Asynchronous, blocking, buffered (user provided)
9 int MPI_Bsend(const void *buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm)
10 // Asynchronous, non-blocking, buffered (user provided)
11 int MPI_Ibsend(const void *buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm, MPI_Request *request)
12
13 // Synchronous, blocking, not buffered
14 int MPI_Ssend(const void *buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm)
15 int MPI_Rsend(const void *buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm)
16
17 // Synchronous, non-blocking, typically not buffered
18 int MPI_Issend(const void *buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm, MPI_Request *request)
19
20 // Usage
21 MPI_Request reqs[2];
22 MPI_Irecv(&data_in, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, &reqs[0]);
23 MPI_Isend(&data_out, 1, MPI_INT, next, 0, MPI_COMM_WORLD, &reqs[1]);
24 MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);

```

4.3 Group Operations

```

1 int MPI_Comm_group(MPI_Comm comm, MPI_Group *group)
2 int MPI_Group_size(MPI_Group group, int *size)
3 int MPI_Group_rank(MPI_Group group, int *rank)
4 int MPI_Group_union(MPI_Group group1, MPI_Group group2, MPI_Group *newgroup)
5 int MPI_Group_intersection(MPI_Group group1, MPI_Group group2, MPI_Group *newgroup)
6 int MPI_Group_difference(MPI_Group group1, MPI_Group group2, MPI_Group *newgroup)
7 int MPI_Group_incl(MPI_Group group, int n, const int ranks[], MPI_Group *newgroup)
8 int MPI_Group_excl(MPI_Group group, int n, const int ranks[], MPI_Group *newgroup)
9 int MPI_Group_compare(MPI_Group group1, MPI_Group group2, int *result)
10 int MPI_Group_free(MPI_Group *group)

```

4.4 Communicators

```
1 int MPI_Comm_size(MPI_Comm comm, int *size)
2 int MPI_Comm_rank(MPI_Comm comm, int *rank)
3 int MPI_Comm_create(MPI_Comm comm, MPI_Group group, MPI_Comm *newcomm)
4 int MPI_Comm_compare(MPI_Comm comm1, MPI_Comm comm2, int *result)
5 int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm)
6 int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm *newcomm)
7 int MPI_Comm_free(MPI_Comm *comm)
8 int MPI_Intercomm_create(MPI_Comm local_comm, int local_leader, MPI_Comm peer_comm,
    int remote_leader, int tag, MPI_Comm *newintercomm)
9
10 // Usage
11 int row_color = rank / row_size;
12 MPI_Comm row_comm;
13 MPI_Comm_split(MPI_COMM_WORLD, row_color, rank, &row_comm);
```

4.5 Virtual Topologies

```
1 int MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[], const int periods
    [], int reorder, MPI_Comm *comm_cart)
2 int MPI_Cart_get(MPI_Comm comm, int maxdims, int dims[], int periods[], int coords[])
3 int MPI_Cartdim_get(MPI_Comm comm, int *ndims)
4 int MPI_Cart_coords(MPI_Comm comm, int rank, int maxdims, int coords[])
5 int MPI_Cart_rank(MPI_Comm comm, int coords[], int *rank)
6 int MPI_Cart_shift(MPI_Comm comm, int direction, int disp, int *rank_source, int *
    rank_dest)
7
8 int MPI_Graph_create(MPI_Comm comm_old, int nnodes, const int index[], const int edges
    [], int reorder, MPI_Comm *comm_graph)
9 int MPI_Graph_get(MPI_Comm comm, int maxindex, int maxedges, int index[], int edges[])
10 int MPI_Graphdims_get(MPI_Comm comm, int *nnodes, int *nedges)
11 int MPI_Graph_neighbors_count(MPI_Comm comm, int rank, int *nneighbors)
12 int MPI_Graph_neighbors(MPI_Comm comm, int rank, int maxneighbors, int neighbors[])
13
14 // Usage
15 int dims[2] = {4,4}, periods[2] = {0,0};
16 MPI_Comm cart;
17 MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart);
18 int up, down;
19 MPI_Cart_shift(cart, 0, 1, &up, &down); // 0 = row axis, 1 = displacement
```

4.6 Synchronisation

```
1 int MPI_Barrier(MPI_Comm)
2 int MPI_Ibarrier(MPI_Comm comm, MPI_Request *request)
```

4.7 Collective Communication

```
1 int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
2 int MPI_Allgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *
    recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm)
3 int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *
    recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)
4 int MPI_Scatterv(const void *sendbuf, const int sendcounts[], const int displs[],
    MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int
    root, MPI_Comm comm)
```

```

5  int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *
    recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)
6  int MPI_Gatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *
    recvbuf, const int recvcounts[], const int displs[], MPI_Datatype recvtype, int
    root, MPI_Comm comm)
7  int MPI_Reduce(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype,
    MPI_Op op, int root, MPI_Comm comm)
8  int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype
    , MPI_Op op, MPI_Comm comm)
9  int MPI_Reduce_scatter(const void *sendbuf, void *recvbuf, const int recvcounts[],
    MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
10 int MPI_Alltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *
    recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm)
11 int MPI_Alltoallv(const void *sendbuf, const int sendcounts[], const int sdispls[],
    MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int rdispls[],
    MPI_Datatype recvtype, MPI_Comm comm)
12
13 // Usage
14 MPI_Bcast(&config, 1, MPI_INT, 0, MPI_COMM_WORLD);
15 MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
16
17 // rcount is size of chunk each process receives
18 MPI_Scatter(big_arr, chunk, MPI_INT, my_chunk, chunk, MPI_INT, 0, MPI_COMM_WORLD);

```

4.8 Datatypes

```

1  MPI_CHAR
2  MPI_SHORT
3  MPI_INT
4  MPI_LONG
5  MPI_UNSIGNED_CHAR
6  MPI_UNSIGNED_SHORT
7  MPI_UNSIGNED_LONG
8  MPI_UNSIGNED
9  MPI_FLOAT
10 MPI_DOUBLE
11 MPI_LONG_DOUBLE
12 MPI_BYTE
13 MPI_PACKED
14
15 int MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype *newtype)
16 int MPI_Type_commit(MPI_Datatype *datatype)
17 int MPI_Type_create_struct(int count, int array_of_blocklengths[], const MPI_Aint
    array_of_displacements[], const MPI_Datatype array_of_types[], MPI_Datatype *
    newtype)
18
19 // Usage
20 MPI_Datatype row_type;
21 MPI_Type_contiguous(N, MPI_DOUBLE, &row_type);
22 MPI_Type_commit(&row_type);
23 MPI_Send(matrix[1], 1, row_type, dest, 0, MPI_COMM_WORLD);

```

4.9 Random Usage Examples

```

1
2 MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
3
4 MPI_Cart_create(

```

```

5     MPI_COMM_WORLD, // default communicator
6     2,              // 2D grid
7     int[]{4, 4},    // 4 rows, 4 columns
8     int[]{0, 0},    // no wraparounds
9     0,              // no reordering of ranks
10    &cartcomm       // new communicator
11 }
12
13 MPI_Cart_shift(
14     cartcomm, // cartesian communicator
15     0,        // up/down axis
16     2,        // shift
17     &nbrs[UP]  // source, rank directly above
18     &nbrs[DOWN] // destination, rank directly below
19 )
20 #define UP 0, DOWN 1, LEFT 2, RIGHT 3

```

4.10 Command Line

```

1 mpic++ hello.cpp -o hello
2 mpirun -np 4 ./hello
3 srun --nodes 1 --ntasks 4 ./hello
4 srun --nodes 4 --ntasks 4 ./hello
5 srun --nodes 4 --ntasks-per-node 2 ./hello
6 srun -p i7-7700 --nodes 1 --ntasks 8 ./hello
7 srun -p i7-7700 --nodes 1 --ntasks 32 --overcommit ./hello
8 // Block distribution
9 srun -N 3 -n 8 --distribution=block --exclusive ./hello
10 // Cyclic distribution
11 srun -N 3 -n 8 --distribution=cyclic --exclusive ./hello
12 // Plane distribution, fixed blocks of size x, cyclically distributed
13 srun -N 3 -n 8 --distribution=plane=2 --exclusive ./hello

```

4.11 AY24/25 S2 Final

```

1 int main(int argc, char** argv) {
2     int rank, size;
3     MPI_Init(&argc, &argv);
4     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5     MPI_Comm_size(MPI_COMM_WORLD, &size);
6     int num_images;
7     int* images;
8     if (rank == 0) {
9         num_images = get_num_images(...args omitted);
10        images = read_image_data(num_images, ...args omitted);
11    }
12    MPI_Bcast(&num_images, 1, MPI_INT, 0, comm);
13    int N = (num_images / size);
14    int* images_self = malloc(sizeof(int) * 320 * 320 * N);
15    MPI_Scatter(images, num_images * 320 * 320, MPI_INT, images_self, N * 320 * 320,
16        MPI_INT, 0, MPI_COMM_WORLD);
17    int* images_processed = process(images_self, N);
18    MPI_Gather(images_processed, N * 320 * 320, MPI_INT, images, num_images * 320 *
19        320, MPI_INT, 0, MPI_COMM_WORLD);
20    MPI_Finalize();
21    return 0;
22 }

```

4.12 AY24/25 S1 Final

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define T 100 // Number of time steps
6 #define alpha 0.01 // Diffusion rate
7
8 void update_temperature(double* temp, double* temp_new, int rows, int Ny) {
9     int i, j;
10    // Point A
11    start_i = 1
12    start_j = 1
13    end_i = rows - 1
14    end_j = Ny - 1
15    for (i = start_i; i < end_i; i++) {
16        for (j = start_j; j < end_j; j++) {
17            temp_new[i * Ny + j] = temp[i * Ny + j] + alpha * (
18                temp[(i + 1) * Ny + j] + temp[(i - 1) * Ny + j] +
19                temp[i * Ny + (j + 1)] + temp[i * Ny + (j - 1)] -
20                4 * temp[i * Ny + j]
21            );
22        }
23    }
24 }
25
26 int main(int argc, char** argv) {
27     int rank, size;
28     MPI_Init(&argc, &argv);
29     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
30     MPI_Comm_size(MPI_COMM_WORLD, &size);
31
32     if (argc < 3) {
33         if (rank == 0) {
34             fprintf(stderr, "Usage: %s Nx Ny\n", argv[0]);
35         }
36         MPI_Finalize();
37         exit(EXIT_FAILURE);
38     }
39
40    // Read Nx, Ny from command line
41    int Nx = atoi(argv[1]);
42    int Ny = atoi(argv[2]);
43
44    // set chunk size and other variables
45    // Point B
46    int rows_per_rank = Nx / size;
47    int rows = rows_per_rank + 2;
48
49    // Allocate temperature arrays
50    double* temp = (double*)calloc(rows * Ny, sizeof(double));
51    double* temp_new = (double*)calloc(rows * Ny, sizeof(double));
52
53    // Set initial temperature (heat source in the center rank)
54    if (rank == size / 2) {
55        temp[(rows_per_rank / 2) * Ny + Ny / 2] = 100.0;
56    }
57
58    // Point C
```

```

59 // Nothing
60
61 // Main simulation loop
62 for (int t = 0; t < T; t++) {
63     // Boundary exchange with neighboring ranks
64     // Point D
65     // Ignore boundaries...
66     int up = rank - 1;
67     int down = rank + 1;
68     // Deadlock solution
69     // If even rank, send then receive
70     // If odd rank, receive then send
71     MPI_Send(&temp[1 * Ny], Ny, MPI_DOUBLE, up, 0, MPI_COMM_WORLD);
72     MPI_Recv(&temp[(rows - 1) * Ny], Ny, MPI_DOUBLE, down, 0, MPI_COMM_WORLD,
73             MPI_STATUS_IGNORE);
74
75     MPI_Send(&temp[(rows - 2) * Ny], Ny, MPI_DOUBLE, down, 1, MPI_COMM_WORLD);
76     MPI_Recv(&temp[0 * Ny], Ny, MPI_DOUBLE, up, 1, MPI_COMM_WORLD,
77             MPI_STATUS_IGNORE);
78
79     // Update temperature within the rank's subgrid
80     update_temperature(temp, temp_new, rows, Ny);
81
82     // Swap pointers for the next iteration
83     double* temp_swap = temp;
84     temp = temp_new;
85     temp_new = temp_swap;
86 }
87
88 // Get results to root rank
89 double* final_grid = NULL;
90 if (rank == 0) {
91     final_grid = (double*)malloc(Nx * Ny * sizeof(double));
92 }
93
94 // Point E
95 MPI_Gather(&temp[Ny], rows_per_rank * Ny, MPI_DOUBLE, final_grid, rows_per_rank *
96           Ny, MPI_DOUBLE, 0, MPI_COMM_WORLD);
97
98 // Print final grid (only by root rank)
99 if (rank == 0) {
100     for (int i = 0; i < Nx; i++) {
101         for (int j = 0; j < Ny; j++) {
102             printf("%f", final_grid[i * Ny + j]);
103         }
104         printf("\n");
105     }
106     free(final_grid);
107 }
108
109 // Clean up
110 free(temp);
111 free(temp_new);
112 MPI_Finalize();
113 return 0;
114 }

```